고객지원 에이전트(support) 구성안

> 연동 문서: 마케팅 ↔ 고객지원 대외 소통 루프

멀티 에이전트 AX 플랫폼 「팀 공통 규약」(2026-10-21)에 맞춰 고객지원 부서 에이전트와 오케스트레이션 대시보드를 구성하는 계획이다. 담당: 병환 (부서 에이전트 `support` + 공통 모듈 「오케스트레이션 대시보드·문의 위젯」).

이 문서는 **규약을 우리 부서에 적용한 초안**이다. 규약에서 "제안"인 항목은 이 문서에서도 제안으로 두었고, 팀 합의가 나면 확정으로 바꾼다. 현재 저장소(`bxbae/iTshirt-cat`)의 캣북스 프로토타입이 이미 해 주는 것과 새로 해야 하는 것을 구분해 적었다.

표기: **[기존 규칙]**과 **[제안]**은 규약의 상태 그대로이고, **[현재 코드]**는 프로토타입에서 이미 되는 것, **[갭]**은 규약과 다른 점이다.

---

1. 한눈에 보기

항목	내용
에이전트 이름	`support`
담당	병환
한 줄 역할	고객 문의를 받아 **영수증과 정책으로 환불·보상을 판정**하고, 반품 입고 순서로 지급하며, 답변 초안을 근거와 함께 만든다
부서 그림에서의 위치	고객지원 — 문의 응대, 보상 계산. 마케팅·상품MD와 "대외 소통"으로 직통 연결
공통 모듈	오케스트레이션 대시보드 + 문의 위젯
첫 고객사	판매 사이트(가상 오픈마켓 / 도서·문구, 규약 "확인 필요 4")

강점과 갭 요약

구분	내용
이미 규약에 맞는 것	숫자는 코드가 계산하고 LLM은 문장만 쓴다. 모든 판정에 근거(규칙 번호·정책 문서 번호)와 감사 기록이 남는다. LLM 실패 시 규칙 기반(mock)으로 대체된다. 외부 입력(문의)의 개인정보 마스킹과 발송 전 가드가 있다
크게 바꿔야 하는 것	① A2A 에이전트 형태(Agent Card, Task, contextId) ② 상태를 바꾸는 모든 지급을 `execute()` 관문으로 ③ 주문·상품 데이터를 직접 읽지 않고 판매 사이트 API/MCP로 ④ 화면을 React 19 + Vite + common-frontend로 ⑤ 사람 승인(상담원) 구조를 C 등급 "보류 + 리포트"와 맞추기

---

2. 규약 18항목 적용표

#	항목	규약	고객지원에서 할 일	현재 상태
1	레포 구성	레포 1개, README 5항목, CLAUDE.md, .env.example **[기존 규칙]**	모노레포 `ax-backend/agents/support/` 에 **독립 패키지**로 둔다(자체 pyproject·README·테스트·Dockerfile, 다른 부서 import 금지). 배치는 마케팅↔고객지원 루프 6장 **[제안]**. README 5항목은 6장	[현재 코드] `.env.example` 있음. 쇼핑몰·콘솔이 한 레포에 섞여 있음 → 분리 필요
2	스택	Python 3.12 + FastAPI + a2a-sdk, 에이전트 1개 = 컨테이너 1개 **[제안]**	a2a-sdk로 Agent Card·Task 엔드포인트 추가	[현재 코드] Python 3.12 + FastAPI + 컨테이너. a2a-sdk 없음
3	화면 틀	common-frontend를 버전 태그로 설치, AppShell 메뉴에 부서 항목, 색은 토큰만 **[기존 규칙]**	대시보드·콘솔을 common-frontend 메뉴 "고객지원" 아래로 이식	[갭] 현재는 Next.js 14 + 자체 Tailwind 테마. React 19 + Vite로 다시 옮겨야 함
4	인증	사람은 JWT 하나, 에이전트는 에이전트별 토큰, 요청자는 토큰에서 추출 **[제안]**	템플릿의 인증 모듈 사용	[갭] 현재는 `X-User-Id` 헤더와 열린 `/api/admin` (데모용)
5	고객사 구분	모든 테이블·요청·Task에 `tenant_id` **[제안]**	클레임·문의·감사 로그 테이블에 `tenant_id` 컬럼	[갭] 없음. 규약 "확인 필요 7"에서 빼기로 하면 생략
6	API 형식	`/api/<부서>/<복수 명사>`, 성공 `{data, meta}` / 실패 `{error: {code,message}}`, 시간은 UTC 저장·KST 표시 **[제안]**	7장 경로 표대로 이전	[현재 코드] 시간은 UTC 저장·KST 표시 이미 지킴. 경로·응답 봉투는 다름
7	부서 간 통신	A2A(일 맡기기) / MCP(도구 쓰기), 모든 부서 직통 **[제안]**	8장 통신 지도	[갭] 현재는 단일 앱 안의 함수 호출
8	오케스트레이터	통합 에이전트가 사건 판단·분배·취합, 이관 큐 **[제안]**	통합 에이전트가 보내는 요청을 받는 쪽으로만 동작	해당 없음(준하 담당)
9	사건 묶음	모든 요청·로그에 `contextId`, 사건 1개 = 1개	클레임·문의·감사 로그에 `context_id` 필수	[갭] 없음. 문의(conversation)와 클레임이 `order_id`로만 느슨하게 연결
10	데이터 공유	DB 직접 읽기 금지, A2A·API·MCP로만. 주문·결제·상품 원천은 판매 사이트 **[제안]**	주문·영수증은 판매 사이트 MCP로 조회	[갭] 현재는 같은 DB의 `orders`를 직접 읽음. 가장 큰 구조 변경
11	상태 변경 실행	모두 `execute()` 관문을 거쳐 등급 판정 후 실행 **[제안]**	지급·환불·후기 숨김을 관문으로 감쌈	[갭] `claims.execute()`가 직접 지급
12	자율성 등급	A/B/C × L3/L2/L1, 착수 전 액션 등급 신고 **[제안]**	5장 신고표	[갭] 자동 처리 한도(100,000원)만 있고 등급 개념 없음

#	항목	규약	고객지원에서 할 일	현재 상태
13	근거·기록	결과마다 쿼리·데이터·참조 로그, 감사 로그, 근거 없는 값은 지어내지 않음 **[기존 규칙]**	판정 결과에 `근거` 필드로 첨부	[현재 코드] 판정 이유·정책 번호·감사 기록 있음. 영수증 원본 스냅샷 첨부는 추가 필요
14	계산	숫자는 SQL·코드, LLM은 해석·판단만 **[제안]**	유지	[현재 코드] 환불액·보상액·순서는 모두 코드. 가드가 LLM 문장의 금액을 근거와 대조
15	MCP	공통 봉투 `{ok, collected_at, data, truncated}`, 목 모드, 이름 `<대상>_<동사>_<대상>` **[기존 규칙]**	판매 사이트용 MCP 클라이언트(조회), 목 모드로 단독 개발	[현재 코드] mock LLM 모드는 있음. MCP 없음
16	LLM	기획서는 Claude, DejaFix는 OpenAI 호환 + 실패 시 규칙 기반 **[제안]**	제공자 인터페이스 유지. 확인 필요 3 결정에 따라 교체	[현재 코드] Anthropic + mock 대체. OpenAI 호환 클라이언트는 없음
17	이름·설정	환경 변수 부서 접두사 대문자, 테이블 복수형 snake_case, 포트 대역 **[제안]**	`SUPPORT_` 접두사, 7장 이름 변경	[갭] 현재 `LLM_PROVIDER` 등 접두사 없음. `audit_log` → `audit_logs` 등
18	Git·비밀값	main·dev, Conventional Commits, PR 리뷰 1명, 비밀값은 .env **[기존 규칙]**	그대로	[갭] 현재 한글 커밋 메시지. 규칙에 맞춰 영문 접두사 사용

---

3. 부서 구성에서 고객지원의 몫

그림(조직도)의 고객지원 연결 `문의 응대`, `보상 계산`을 skill로 옮긴다. 규약 "확인 필요 1"에 따라 마켓 운영은 재무/정산으로 흡수되므로, **사기·가품 감시와 판매자 심사는 하지 않는다.** 다만 고객 신고 접수는 우리 쪽에서 받아 재무/정산에 넘긴다.

하는 것	하지 않는 것 (README의 "하지 않는 것")
문의 분류·답변 초안·발송 전 검증	주문·결제·상품 원천 데이터 관리 (판매 사이트)
영수증·정책 기반 클레임 판정, 보상액 계산	판매자 정산·손실 집계·금액 기준표 관리 (재무/정산)
반품 입고 순 지급 큐	공지 게시·배너 (마케팅)
후기 자동 보류와 선제 연락 대상 표시	티켓 생성·배포 추적 (기획개발)
사건 기록 요약을 인사 에이전트에 전달	회사 기억 저장·검색 (데이터분석 모듈)
오케스트레이션 대시보드, 문의 위젯	에이전트 간 분배·취합 (통합 에이전트)

---

4. Agent Card (초안)

`/.well-known/agent-card.json`으로 공개할 내용이다. 필드 이름과 A2A 메시지 안의 위치는 공통 템플릿과 함께 확정한다.

```
{
  "name": "support",
  "owner": "병환",
  "description": "문의 응대, 영수증·정책 기반 클레임 판정, 반품 입고 순 보상 지급",
  "url": "<A2A 요청 주소, 포트 대역 합의 후 확정>",
  "skills": [
    {
      "id": "claim_evaluate",
      "description": "주문과 신청 내용을 영수증·정책으로 대조해 결과(자동 승인/자료 요청/상담 확인/자동 거절)와 금액을 계산한다. 상태를 바꾸지 않는다.",
      "input": { "context_id": "str", "tenant_id": "str", "order_id": "str", "claim": "ClaimInput", "untrusted": { "customer_text": "str" } },
      "output": { "outcome": "enum", "amounts": "Amounts", "reasons": "Reason[]", "evidence": "Evidence" },
      "action_grade": null
    }
  ],
  "acl": "5장 표 참고"
}
```

skills 전체는 다음 장 표에 있다.

---

5. skill과 액션 등급 신고표 [제안]

규약: "각 부서는 자기 에이전트가 하는 액션의 등급을 개발 착수 전에 신고한다", "등급은 규칙(기준표)이 정하고 LLM이 정하지 않는다", "등록되지 않은 액션은 C". 아래는 신고 초안이다. **금액 기준은 재무/정산(태윤)의 기준표가 정하며**, 여기 숫자는 현재 프로토타입 값이다.

5-1. 읽기 skill (상태 변경 없음 → 등급 없음, 근거 첨부 필수)

skill	하는 일	요청 가능 에이전트(ACL, 제안)
`claim_evaluate`	영수증·정책 대조 판정. 같은 입력이면 같은 결과	통합, 재무/정산, 마케팅
`compensation_calculate`	지연·불량 등 보상액 계산(코드)	통합, 재무/정산
`inquiry_draft`	문의 분류, 정책 검색, 답변 초안 3종 + 가드 결과	통합
`queue_status`	보상 큐 순서·조건·막힌 사유 조회	통합, 재무/정산, 데이터분석

5-2. 상태를 바꾸는 skill

skill	하는 일	등급	처리	이유
`reply_send`	가드를 통과한 답변을 고객에게 발송 (금전 약속 없음)	A (L3)	바로 실행	되돌릴 수 있고 약속이 없음

skill	하는 일	등급	처리	이유
`reply_send` (금전 약속 포함)	근거 있는 환불·보상 안내 포함	B (L2)	재무/정산이 2차 확인	약속은 지급과 같은 무게
`claim_register`	신청 접수·자료 요청·철회 기록	A	바로 실행	기록성
`review_moderate`	후기 보류·숨김·공개	A	바로 실행	되돌릴 수 있음
`case_record`	상담 요약을 인사 에이전트에 전달	A	바로 실행	기록성
`cancel_refund_pay`	출고 전 취소 환불	B	재무/정산 2차 확인 후 실행	돈이 나감
`compensation_pay`	불량·지연 보상 적립금 지급, 반품 환불	B (기준 금액 이상은 C)	B: 교차검증 후 실행 / C: 보류 + 근거 리포트	규약 예시 "쿠폰 지급 = B, 기준 금액 이상 보상 = C"
`return_queue_process`	입고 순서대로 위 지급을 일괄 실행	건별 `compensation_pay` 등급을 따름	건마다 관문 통과	일괄 처리가 등급을 우회하지 못하게
등록되지 않은 액션	—	C	보류	규약

5-3. 프로토타입 값을 기준표로 옮길 대상

현재 상수	값	기준표에서 맡을 곳
자동 처리 한도	100,000원	B/C 경계 후보 (재무/정산 결정)
불량 보상 적립금	2,000원	보상 정책표
지연 보상	3일 1,000원, 7일 3,000원	보상 정책표
반품 배송비 / 무료배송 기준	3,000원 / 15,000원	판매 사이트 정책 (우리는 조회만)
누적 한도	없음	**신규**: contextId별·시간당 지급 누적 한도 (권장 안전장치)

---

6. README 5항목 초안

1. **역할**: 고객 문의와 환불·보상 신청을 영수증·정책으로 판정하고, 반품 입고 순서로 지급하며, 답변 초안을 근거와 함께 만든다.
2. **실행 방법**: `docker compose up` (에이전트 1컨테이너 + 대시보드), 목 모드는 `SUPPORT_MOCK=1`. 판매 사이트 MCP가 없어도 목 데이터로 단독 동작한다.
3. **입출력 계약**: 4·5장 skill 표, Task 확장 필드(9장), 응답 봉투 `{data, meta}`.
4. **의존 레포**: 공통 템플릿(준하), common-mcp(판매 사이트 조회·지급 도구), common-frontend, 안전 레이어(태윤, `execute()`), 판매 사이트(재헌).
5. **하지 않는 것**: 3장 오른쪽 열.

---

7. API와 이름 정리 (현재 → 목표)

현재	목표
`POST /api/claims`, `GET /api/claims`	`/api/support/claims`
`POST /api/claims/preview`	`/api/support/claims/previews`
`GET/POST /api/admin/claims…`	`/api/support/claims/{id}/…` (인증 필요)
`/api/admin/queue`, `/api/admin/queue/process`	`/api/support/queue-entries`, `POST /api/support/queue-runs`
`/api/inquiries`, `/api/admin/conversations`	`/api/support/inquiries`
`/api/admin/reviews`	`/api/support/reviews`
`/api/admin/dashboard`	`/api/support/dashboards` 또는 오케스트레이션 대시보드 API
응답: 객체/배열 그대로	성공 `{data, meta}`, 실패 `{error:{code,message}}`
환경 변수 `LLM_PROVIDER`, `ANTHROPIC_*`, `AGENT_NAME` …	`SUPPORT_LLM_PROVIDER`, `SUPPORT_ANTHROPIC_*` …
테이블 `audit_log`, `point_ledger`	`audit_logs`, `point_ledgers` (+ `tenant_id`, `context_id`)

판매 사이트가 소유해야 하는 테이블(`users`, `products`, `orders`, `order_items`, `reviews`, 적립금 잔액)은 이 레포에서 빠지고 MCP/API 호출로 바뀐다. 고객지원이 자기 것으로 갖는 테이블은 `claims`, `audit_logs`, `conversations`, `messages`, `feedback_events`, 큐 상태다.

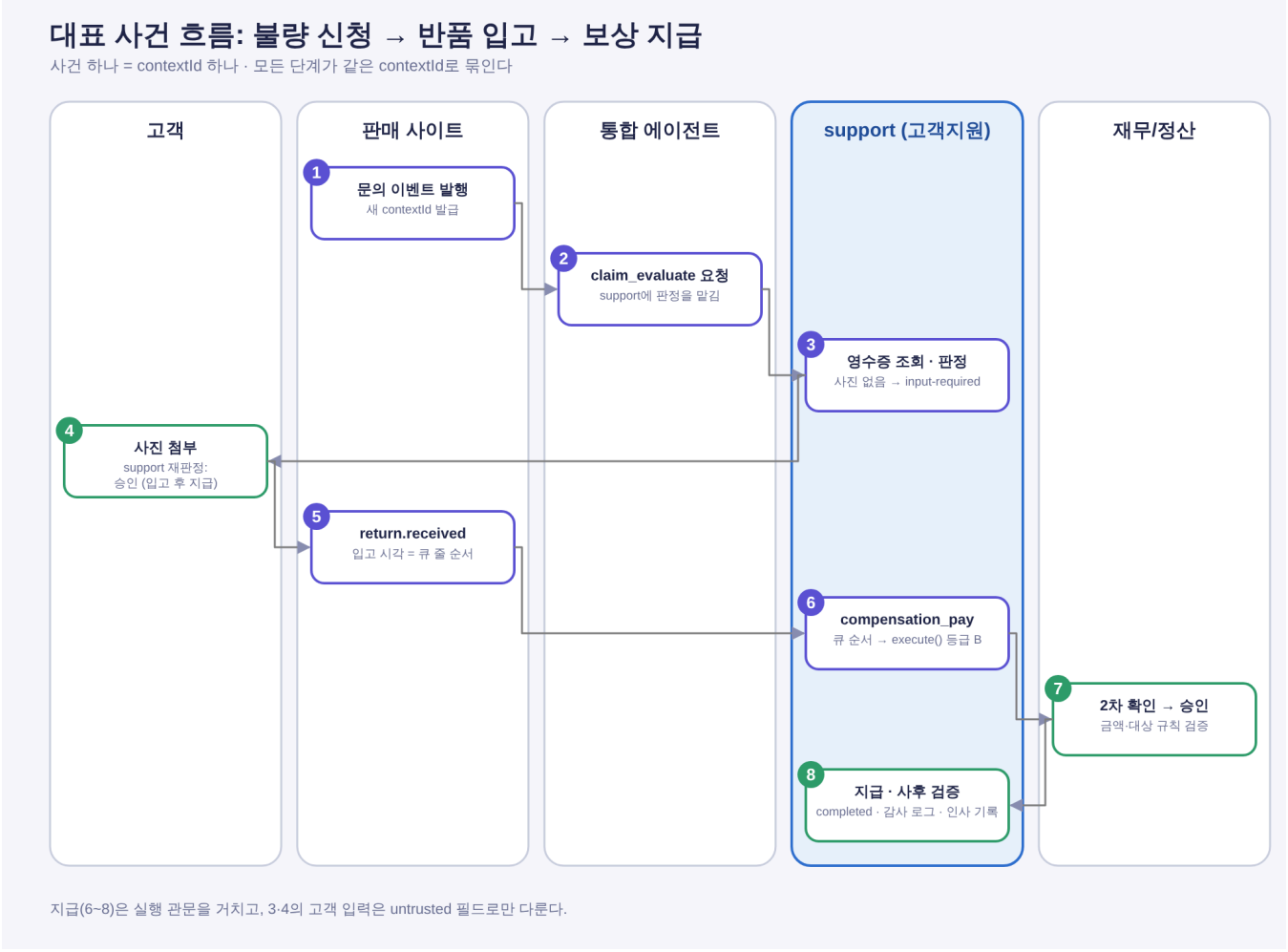
---

8. 통신 지도

```
판매 사이트 ──이벤트(문의 접수·주문·반품 입고)──▶ 통합 에이전트 ──A2A──▶ support
                                                                          │
        MCP(조회): shop_get_order, shop_list_orders                       │
        MCP(실행): shop_credit_points, shop_cancel_order  ◀── execute() 관문
                                                                          │
        A2A(요청): finance  ← compensation_approve / loss_aggregate
                   marketing ← notice_publish        (문의 급증·대규모 이슈 공지)
                   planning  ← ticket_create         (불량 패턴이 버그일 때)
                   hr        ← record_case           (사건 기록, 인수인계)
        A2A(수신): data(분석) → support  이상 탐지 알림, 영향 분석 요청
                   marketing → support   공지 후 문의 대응 요청
```

- 모든 부서 직통을 허용하되 안전장치는 템플릿에 내장한다 (규약).
- `shop_*` 도구 이름은 규약의 `<대상>_<동사>_<대상>` 형식 **제안**이다. 실제 이름과 봉투는 common-mcp와 판매 사이트(재헌) 규격이 확정되면 맞춘다.
- 판매 사이트에는 **`return.received` (반품 입고) 이벤트**가 필요하다. 보상 큐의 줄 순서가 입고 시각이기 때문이다 → 재헌의 이벤트 스키마에 요청.

대표 사건 흐름 (데모 시나리오 후보)



> 고객이 "받은 책 표지가 찢어졌다"고 신청 → 반품 입고 → 보상 지급

단계	누가	무엇	contextId
1	판매 사이트	문의 이벤트 발행	새로 발급
2	통합 에이전트	support에 `claim_evaluate` 요청	동일
3	support	영수증 조회(MCP) → 판정: 사진 없음 → `input-required`	동일
4	고객	사진 첨부 → support가 재판정: 승인(입고 후 지급)	동일
5	판매 사이트	`return.received` 이벤트 → support 큐에 등록	동일
6	support	큐 순서가 되면 `compensation_pay` 요청 → `execute()` 등급 B	동일
7	재무/정산	2차 확인(금액·대상 규칙 검증) → 승인	동일
8	support	지급 실행(MCP) → 사후 검증 → `completed`, 감사 로그, 인사에 기록	동일

---

9. Task 규약 적용

9-1. 상태 매핑

우리 클레임 상태	A2A Task 상태	비고
접수 직후	`submitted`	
판정·계산 중, 반품 입고 대기, 큐 대기	`working`	
사진 필요 (`needs_info`)	`input-required`	규약은 "다른 부서의 판단"을 예로 듦. **고객 입력도 포함할지 확인 필요**
상담원 확인 (`needs_review`)	`input-required` (재무/정산 판단) 또는 C 보류	사람 승인을 없애는 원칙과 충돌, 아래 확인 필요 2
처리 완료	`completed`	
정책에 따른 **자동 거절**	`completed` (결과 `outcome: auto_rejected`)	아래 주의 참고
권한·형식·루프 위반	`rejected`	규약 정의 그대로
마감 초과	`canceled`	
실행 오류	`failed`	

> **주의**: 규약의 `rejected`는 "에이전트가 요청을 거부(권한·형식·루프)"한 것이다. 환불 신청을 **정책에 따라 거절**한 것은 요청을 정상 처리한 결과이므로 `completed`에 결과 값으로 담는다. 둘을 섞으면 이관률과 실패율 지표가 오염된다.

9-2. 확장 필드와 우리 쪽 대응

필드	고객지원에서의 쓰임
`contextId`	사건 단위. 문의·클레임·감사 로그·큐 항목 모두에 저장
요청자	토큰에서 추출. 본문의 `user_id` 같은 필드는 신뢰하지 않음
경로	이미 `support`가 경로에 있으면 루프 거부
마감 시각	지난 요청은 실행하지 않고 보고. 반품 입고 대기처럼 긴 흐름은 Task를 쪼개 각 단계에 마감을 둠
멱등성 키	`contextId + 액션 + 대상` (예: `ctx-1 + compensation_pay + claim-17`). 현재의 "중복 신청 거절"(수량 기준)과 별개로 필요
근거	영수증 스냅샷 + 규칙 번호 + 정책 문서 번호 + 계산식. 현재 판정 결과에 대부분 있음
`untrusted`	**고객 문의 원문, 후기, 상세 내용**을 여기에만 담고 지시로 해석하지 않음
`tenant_id`	5장 확인 필요 7에 따라

---

10. 안전장치 적용 현황

규약 "필수" 항목 기준이다. 대부분은 공통 템플릿·안전 레이어가 제공하므로 우리는 **쓰는 쪽에서 지킬 것**과 **이미 있는 것**을 구분한다.

안전장치	현재 코드	해야 할 일
실행 관문 `execute()`	없음. `claims.execute()`가 직접 지급	지급·환불·후기 숨김 호출을 관문 뒤로 이동
contextId 필수	없음	모든 테이블·로그에 추가, 없으면 요청 거부
루프 거부·3홉	없음	템플릿 제공
타임아웃	LLM 호출 90초 제한만 있음	마감 시각 필드 준수
구조화 출력 검증	LLM JSON을 파싱·정규화하고 실패 시 재요청 (**부분**)	스키마(예: pydantic) 검증으로 격상
계산은 코드로	**충족** (판정·금액·순서 모두 코드, 가드가 LLM 문장을 재검증)	유지
근거 첨부	**대부분 충족**	영수증 원본 스냅샷을 결과에 포함
멱등성 키	수량 기준 중복 방지만 있음	`contextId+액션+대상` 키로 지급 중복 거부
사후 검증 (B)	없음	지급 후 잔액·주문 상태를 조회해 의도와 일치하는지 확인
에이전트 인증	없음	템플릿 제공
skill 권한표(ACL)	없음	Agent Card의 `acl`을 관문이 확인
신뢰 경계	문의에 개인정보 마스킹과 발송 전 가드는 있음. **원문이 프롬프트에 그대로 들어감**	원문을 `untrusted` 구획으로 분리하고 시스템 지시와 섞이지 않게 프롬프트 재구성. 인젝션 시도 테스트 추가
fail-closed	없음	관문 무응답 시 지급 거부
누적 한도 (권장)	없음	고객·contextId·시간당 지급 누적 한도
요청 예산 (권장)	없음	contextId별 LLM 호출 수 상한
규칙 기반 검증 병행 (권장)	**충족** (판정 엔진이 코드 규칙)	유지

---

11. 오케스트레이션 대시보드 + 문의 위젯 (공통 모듈, 병환)

대시보드가 보여 줄 것

규약의 "이관 건수·경로", "등급별 처리·차단 건수", "감사 로그"를 사람이 보는 화면이다.

화면	내용	데이터 출처
사건 목록·타임라인	contextId별로 부서 Task가 어떻게 이어졌는지, 상태, 소요 시간	통합 에이전트의 Task 기록
이관	이관 건수, 경로, 반복되는 경로(= 고쳐야 할 skill 설명)	이관 큐
등급 현황	A/B/C 처리 건수, 차단 건수, C 보류 목록과 근거 리포트	안전 레이어 / 감사 로그
요청 예산	contextId별 요청 수·LLM 호출 수	안전 레이어
고객지원 탭	현재 콘솔(문의함·코파일럿)·클레임 판정·보상 큐·후기 관리	support API

재사용 가능한 것

현재 `/admin` 화면(대시보드, 상담 콘솔, 클레임 판정, 보상 큐, 후기 관리)이 "고객지원 탭"의 기능 원형이다. 컴포넌트(차트, 표, 판정 근거 패널, 큐 목록)는 React로 옮기기 쉬운 구조이지만, Next.js 라우팅과 자체 테마는 common-frontend 토큰으로 바꿔야 한다.

문의 위젯

고객이 판매 사이트 안에서 문의하는 창이다. 현재 `/support`(1:1 문의)와 주문 상세의 신청 화면이 원형이다. - 판매 사이트에 임베드 가능한 형태(독립 번들)로 만들고, 문의를 판매 사이트 이벤트로 발행한다. - 고객 로그인은 판매 사이트의 것을 따른다(인증 항목 4).

---

12. 공통 산출물 체크리스트

산출물	상태	내용
Agent Card + 액션 등급 신고	초안 완료 (4·5장)	규격 고정 후 확정
단독 테스트 시나리오 1개	초안	아래 참고
데모 시나리오의 자기 구간	초안 (8장)	문의 접수 ~ 보상 지급
발표 슬라이드 1장 (As-Is → To-Be)	초안	아래 참고

단독 테스트 시나리오 (통합 에이전트 없이 A2A 요청만)

1. 테스트 클라이언트가 `claim_evaluate`를 `contextId=test-1`로 요청 (목 모드 영수증)
2. 응답에서 `outcome`, 금액, 규칙 번호, 근거를 검증
3. 같은 입력을 다시 보내 결과가 동일한지 확인 (결정성)
4. 경로에 `support`를 넣어 요청 → 루프로 거부되는지
5. ACL에 없는 요청자 토큰으로 `compensation_pay` → 거부되는지
6. 금액을 기준 이상으로 `compensation_pay` → C 보류와 근거 리포트가 나오는지
7. 같은 멱등성 키로 두 번 → 두 번째는 중복 실행 거부

발표 슬라이드 1장 문구 초안

- **As-Is**: 상담원이 영수증과 규정을 눈으로 대조하고 환불·보상을 수작업으로 처리. 약속이 정책과 어긋나도 보내기 전에 못 막음.
- **To-Be**: 고객지원 에이전트가 영수증·정책으로 판정하고, 근거와 함께 보상 큐에서 입고 순서대로 지급. 금전 처리는 안전 레이어(등급)를 통과하고, 사건 전 과정이 contextId로 추적됨.
- **핵심 수치 후보**: 자동 처리율, 규칙 판정을 뒤집은 비율, 가드가 막은 약속 건수, C 보류 건수.

---

13. 개발 순서 (제안)

규약의 "규격 고정 후 단독 개발" 순서에 맞춘다. 규격이 확정되기 전에도 할 수 있는 것을 앞에 둔다.

단계	할 일	의존
0 (지금)	이 문서 합의, 확인 필요 항목 정리, 프로토타입 유지	없음
1	판정 엔진·보상 큐·가드를 **프레임워크 독립 모듈**로 분리 (DB·HTTP 의존 제거, 주문은 인터페이스로 주입)	없음
2	판매 사이트 조회 인터페이스를 MCP 형태(봉투, 목 모드)로 정의하고 목 구현 작성	common-mcp 규격
3	규격 고정 후: 공통 템플릿 위에 `support` 에이전트 생성, Agent Card, contextId, 인증, 구조화 출력 검증	준하 (템플릿)
4	지급·환불·후기 숨김을 `execute()` 관문 뒤로, 등급·멱등성 키·사후 검증	태윤 (안전 레이어, 기준표)
5	이벤트 연동: 문의 접수, `return.received`	재헌 (이벤트 스키마)
6	화면을 React 19 + Vite + common-frontend로 이식, 대시보드 사건·이관·등급 화면	common-frontend, 통합 에이전트 기록
7	문의 위젯, 단독 테스트 시나리오, 데모 리허설	위 전부

단계 1과 2는 다른 사람의 규격을 기다리지 않고 바로 시작할 수 있다.

---

14. 확인 필요 (고객지원 관점)

규약의 "확인 필요 1~8" 중 우리에게 영향이 큰 것과, 이 문서에서 새로 생긴 것이다.

#	질문	왜 중요한가	제안
1	**C 등급 보류건을 누가 언제 푸는가** (규약 확인 필요 2)	현재 프로토타입은 상담원이 승인·거절한다. 규약 기획서는 "사람 없이 보류 + 리포트", 10/20 결정은 "C는 사람이 처리"다	보류 건 목록과 근거 리포트를 대시보드에 두고, 푸는 주체는 팀 결정
2	상담원 승인(`needs_review`)을 A2A 상태로 어떻게 표현할지	`input-required`는 "다른 부서 판단"으로 정의됨	사람 승인이 있으면 `input-required` + 사유 코드로 표시
3	고객에게 자료를 요청하는 `needs_info`를 `input-required`로 볼지	같은 이유	포함하는 쪽으로 제안
4	금액 기준(B/C 경계, 보상액)을 누가 어디에 두는가 (규약 확인 필요 8)	프로토타입의 상수 5개가 기준표로 옮겨져야 함	재무/정산(태윤) 기준표에 일원화, 우리는 조회
5	보상 적립금·환불 실행의 주체	적립금 잔액은 판매 사이트 데이터. 실행은 판매 사이트 MCP 도구	도구는 판매 사이트(재헌), 호출은 관문을 거쳐 support
6	`return.received` 이벤트 발행 주체와 스키마	보상 큐 순서의 근거	판매 사이트 이벤트 스키마에 추가 요청
7	프론트 스택 (React 19 + Vite) 전환 시점	현재 화면은 Next.js 14	단계 6에서 이식, 그전까지 프로토타입 유지
8	LLM 제공자 (규약 확인 필요 3)	현재 Anthropic + mock 대체. 호출 한도·비용 미정	제공자 인터페이스는 유지하고 결정에 따라 어댑터 교체
9	`tenant_id` (규약 확인 필요 7)	컬럼·인덱스·요청 필드에 영향	가상 회사 하나만 다루면 생략

#	질문	왜 중요한가	제안
10	내부 점검 점수(반복 신청 이력)의 규약상 위치	고객에게 비공개이고 자동 승인을 막는 용도	등급 판정 입력값(규칙)으로 두고, 결과 문구는 중립 유지
11	부서 구성 (규약 확인 필요 1)	마켓 운영이 재무/정산에 흡수되면 고객 신고(사기·가품)를 어디로 넘길지	우리가 접수하고 재무/정산 skill로 A2A 전달

---

15. 현재 프로토타입과의 대응

프로토타입은 `bxbae/iTshirt-cat` 브랜치 `claude/inspiring-newton-m40h9p`에 있다. 목표 구조로 옮길 때 어디가 어디로 가는지 정리한다.

현재 위치	목표
`backend/app/rules/refund.py`, `policy.py`, `abuse.py`	support 에이전트의 `claim_evaluate` 핵심 모듈 (프레임워크 독립)
`backend/app/claims.py`	상태 전이 + 지급은 관문 호출로 교체
`backend/app/queue.py`	`queue_status`, `return_queue_process`
`backend/app/copilot/` (PII, 검색, 가드, LLM)	`inquiry_draft`
`backend/app/kb/*.md`	지식베이스 (정책 숫자는 기준표와 일치 테스트 유지)
`backend/app/shop.py`, `models.py`의 `users`·`products`·`orders`·`reviews`, `api_shop.py`, `frontend/app/(shop)`	**판매 사이트(재헌)로 이관하거나 참조 구현으로만 사용.** support에서는 제거
`frontend/app/admin`, `components/admin`	대시보드 "고객지원 탭" (React + Vite로 이식)
`backend/tests` (70개)	판정·큐·가드 테스트는 그대로 가져가고, 관문·ACL·멱등성 테스트를 추가

가져가는 것의 검증 상태: 판정 규칙·큐·가드는 단위·API 테스트와 브라우저 시나리오로 확인했다. Claude 답변 생성은 실제 API로 실행해 보지 못했다.

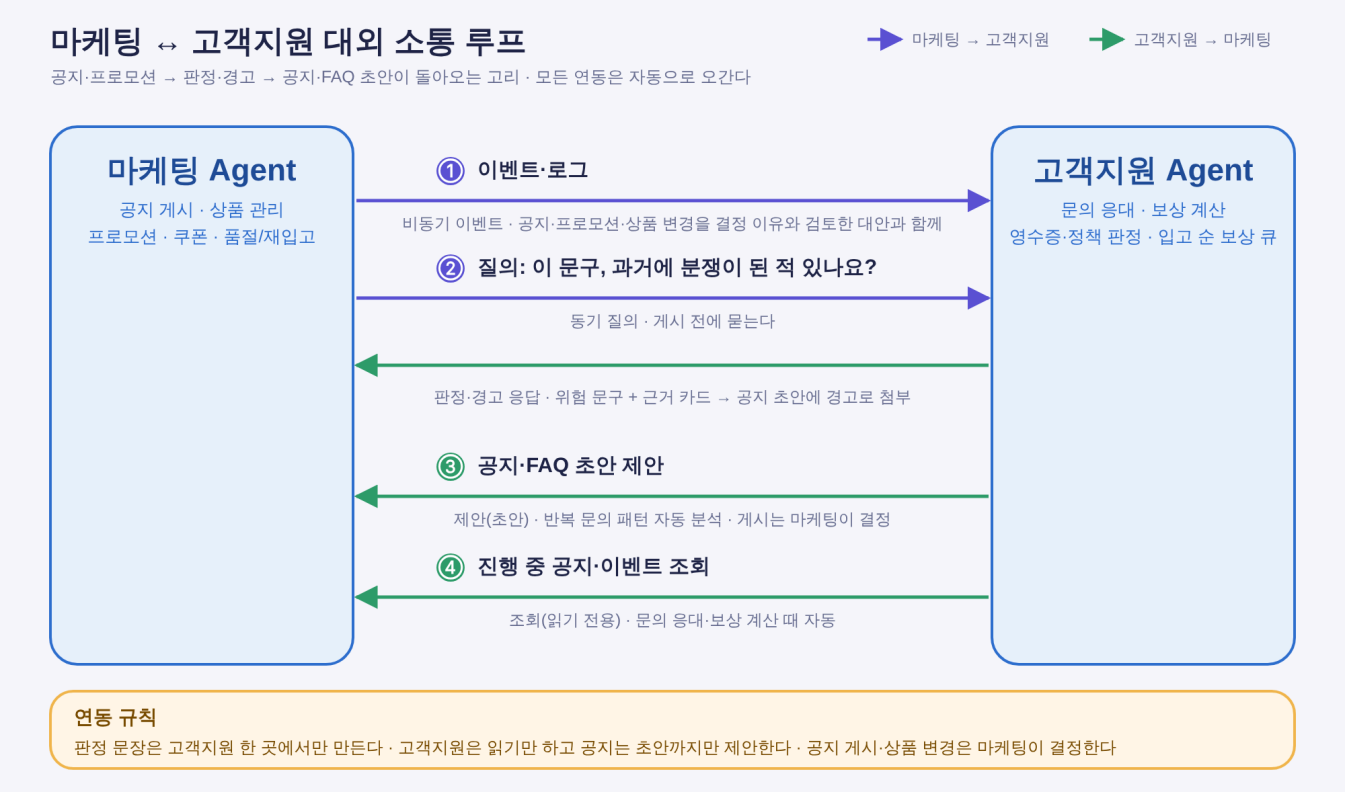
마케팅 ↔ 고객지원 대외 소통 루프

「기획개발 ↔ 인사운영 개선 루프」와 같은 구조로 마케팅 에이전트(`marketing`, 재헌)와 고객지원 에이전트(`support`, 병환)의 연동을 정리한 문서다. 조직도에서 두 부서를 잇는 점선 "대외 소통"을 구체화한 것이고, 공지·상품 관리(마케팅)와 문의 응대·보상 계산(고객지원)이 서로의 기록을 주고받는 고리다.

상위 문서: 고객지원 에이전트 구성안 · 팀 공통 규약(2026-10-21). 표기: **[제안]** 합의 전, **[규약]** 팀 공통 규약 그대로, **[현재 코드]** 이 저장소 프로토타입에서 이미 되는 것.

---

1. 한눈에 보기



번호	방향	방식	무엇을 주고받나	규약상 종류
① 이벤트·로그	마케팅 → 고객지원	비동기 이벤트	공지 게시·수정·종료, 프로모션·쿠폰 시작·종료, 상품 품절·재입고·가격 변경. **결정 이유와 검토한 대안**을 함께	A2A 이벤트
② 질의	마케팅 → 고객지원	동기 질의	"이 공지·프로모션 문구가 정책과 충돌하거나 과거 분쟁을 부른 적 있나요?" ← 판정·경고와 근거 카드	A2A 요청, 읽기 skill
③ 공지·FAQ 초안	고객지원 → 마케팅	제안(초안)	반복 문의 패턴을 분석해 공지·FAQ 초안을 제안. **생성·게시는 마케팅이 결정**	A2A 요청 (제안 큐)
④ 진행 중 공지·이벤트 조회	고객지원 → 마케팅	조회(읽기 전용)	지금 유효한 공지·프로모션·쿠폰·품절 정보. 문의 응대와 보상 계산 때 자동	MCP 또는 A2A 읽기 skill

연동 규칙 (이 루프의 헌법)

1. **판정 문장은 고객지원 한 곳에서만 만든다.** 공지가 정책·약속과 충돌하는지, 과거 분쟁이 있었는지의 판정은 고객지원이 하고, 마케팅은 그 결과를 첨부만 한다.
2. **고객지원은 읽기만 하고, 공지는 초안까지만 제안한다.** 고객지원이 공지를 직접 게시하거나 상품·쿠폰을 바꾸지 않는다.
3. **공지 게시·상품 변경은 마케팅이 결정한다.** 고객지원의 경고·초안은 판단 재료이고, 마케팅이 무시할 수 있다 (단, 무시한 사실은 기록).
4. **모든 연동은 자동으로 오간다.** 사람이 중간에서 전달하지 않는다. 대신 규약의 안전장치(contextId, 실행 관문, 근거 첨부)가 전부 적용된다.

> 인사운영 루프에서 "판정 문장은 인사운영 한 곳", "티켓 생성은 기획개발이 결정"이던 것과 같은 분업이다. 역할 대응은 다음과 같다.

기획개발 ↔ 인사운영	마케팅 ↔ 고객지원
기획개발: 티켓·배포를 결정, 이벤트·로그를 보냄	마케팅: 공지·상품을 결정, 이벤트·로그를 보냄
인사운영: 기록을 쌓고 판정·경고, 개선 티켓 초안 제안	고객지원: 문의·판정 기록을 쌓고 판정·경고, 공지·FAQ 초안 제안
기각된 안과 근거 카드	과거 분쟁·문의 사례와 근거 카드

---

2. 연동별 상세

① 이벤트·로그 (마케팅 → 고객지원)

마케팅이 공지·프로모션·상품 상태를 바꿀 때마다 고객지원에 알린다. 고객지원은 이를 **공지 카드**로 저장하고, 이후 문의 응대의 근거로 쓴다.

공지 카드 [제안]

```
{
  "context_id": "ctx-notice-2026-1021-001",
  "tenant_id": "<확인 필요 7>",
  "kind": "notice | promotion | coupon | product_change",
  "status": "published | scheduled | ended | canceled",
  "title": "배송 지연 안내와 보상",
  "public_text": "10/20~10/22 출고 주문은 평소보다 1~2일 늦을 수 있습니다.",
  "valid_from": "2026-10-21T00:00:00Z",
  "valid_to": "2026-10-25T00:00:00Z",
  "affects": { "product_ids": [], "categories": ["book"] },
  "promises": [ { "type": "points", "amount": 1000, "condition": "예정일보다 3일 이상 지연" }
],
  "decision_reason": "물류 지연이 3일 이어져 고객 문의가 늘고 있음",
  "alternatives_considered": ["공지 없이 문의별 안내", "전체 쿠폰 지급"],
  "evidence": { "queries": [], "refs": [] },
  "untrusted": { }
}
```

- `promises`가 핵심이다. 마케팅이 고객에게 한 약속을 **구조화된 값**으로 받아야 고객지원이 보상 계산에 정확히 쓸 수 있다. 문장에서 금액을 뽑아내지 않는다 (규약 "숫자는 코드가 산출").
- 고객지원은 `status = published`이고 기간 안인 카드만 응대 근거로 인용한다. 예정·종료·취소 카드는 인용하지 않는다.
- 공지 본문은 외부로 나가는 문구이지만 판매자·고객 입력을 포함할 수 있으므로 **지시로 해석하지 않는 데이터 필드**로만 다룬다 (규약 신뢰 경계).
- [현재 코드] 정책 문서는 `app/kb/*.md` 정적 파일이다. 공지 카드는 **기간이 있는 임시 근거**라서 별도 저장소(공지 카드 테이블)와 검색 대상 추가가 필요하다.

② 질의: 이 문구, 과거에 분쟁이 된 적 있나요? (마케팅 → 고객지원, 동기)

마케팅이 공지·프로모션 **초안을 게시하기 전에** 묻는다. 고객지원은 코드 규칙과 과거 사례로 판정해 경고를 돌려준다.

요청: `notice_check` skill, 입력 `{ context_id, draft: { title, public_text, promises[], valid_from, valid_to }, untrusted }`.

응답 (판정은 항상 이 구조):

```
{
  "verdict": "ok | warn | block",
  "findings": [
    {
      "code": "ABSOLUTE_WORDING",
      "level": "block",
      "message": "'무조건 전액 환불'은 전자책·주문제작 굿즈의 단순변심 환불 불가 규정과 충돌합니다.",
      "evidence": { "policy_refs": ["KB-REF-01#3"], "cases": ["inq-1042", "claim-88"] }
    }
  ],
  "similar_cases": [ { "id": "inq-1042", "summary": "전액 환불 문구를 보고 전자책 환불을 요구",
"outcome": "자동 거절 후 불만" } ],
  "computed": { "max_exposure_points": 12000, "basis": "예상 대상 주문 × 약속 금액 (코드 계산)"
}
}
```

- 판정은 규칙(코드)이 하고, LLM은 설명 문장을 다듬는 데만 쓴다. 노출 금액처럼 숫자는 코드가 계산한다 (규약).
- 마케팅은 이 응답을 **공지 초안에 경고로 첨부**한다. 첨부된 경고를 보고 문구를 고칠지, 그대로 게시할지는 마케팅이 정한다. 단 `block`을 무시하고 게시하면 감사 로그에 남기고 대시보드에 표시한다 [제안].
- [현재 코드] 이미 있는 재사용 자산:
- 답변 가드(`copilot/validate.py`)의 단정적 표현(`무조건`, `100%`, `즉시 환불`), 정책에 없는 금액 약속, 조건부 정책(전자책·주문제작) 검출.
- 정책 수치(`rules/policy.py`)와 규칙→정책 문서 매핑.
- 새로 필요한 것: ① 가드를 "고객 답변"뿐 아니라 "공지 초안"에도 쓰도록 일반화, ② 과거 문의·클레임에서 유사 사례를 찾는 **사례 카드 저장소**(현재 문의는 있지만 검색 가능한 사례 색인이 없음), ③ 약속 금액 × 예상 대상 수 노출 계산.

③ 공지·FAQ 초안 제안 (고객지원 → 마케팅)

고객지원이 문의 패턴을 자동으로 분석해, 마케팅에 공지나 FAQ **초안을 제안**한다. 만들지 말지는 마케팅이 정한다.

감지 규칙(코드) [제안]

패턴	조건 예	제안
카테고리 급증	같은 분류 문의가 최근 24시간에 평소 대비 N배	해당 이슈 공지 초안
반복 질문	같은 정책 조항(`KB-DEL-01#2` 등)을 인용한 문의가 N건 이상	FAQ 초안
약속 오해	같은 공지 카드에서 파생된 자동 거절이 N건 이상	문구 수정 제안 (②와 연결)
상품 문제	같은 상품의 불량 신청이 N건 이상	상품 점검·판매 중지 검토 제안 (마케팅·상품MD와 기획개발 티켓)

제안 초안

```
{
  "context_id": "ctx-support-pattern-2026-1021-003",
  "pattern": "delivery_delay_spike",
  "evidence": { "queries": ["문의 분류별 24시간 집계"], "case_ids": ["inq-1101", "inq-1102"],
"counts": { "now": 37, "baseline": 6 } },
  "draft": { "title": "배송 지연 안내", "public_text": "…", "promises": [] },
  "reason": "배송 문의가 평소의 6배로 늘었고 같은 안내를 반복해서 보내고 있음",
  "caveats": ["보상 약속은 포함하지 않았음. 포함하려면 재무/정산 기준표 확인 필요"]
}
```

- **초안까지만**: 고객지원은 `promises`를 비워 두거나 정책표에 이미 있는 값만 넣는다. 새로운 보상 약속은 제안하지 않는다 (금액 기준은 재무/정산 소관).
- 제안은 마케팅의 **제안 큐**에 들어가고, 마케팅이 채택·보류·기각을 기록한다. 기각 사유는 다음 ② 질의의 재료가 된다 (인사운영 루프의 "기각된 안" 카드와 같은 역할).
- [현재 코드] 대시보드의 분류별 집계(`by_category`)와 가드 적발 통계가 있다. 시간대별 기준선 비교와 제안 생성은 새로 만든다.

④ 진행 중 공지·이벤트 조회 (고객지원 → 마케팅, 읽기 전용)

문의를 응대하거나 보상액을 계산하기 전에 자동으로 마케팅의 현재 상태를 읽는다.

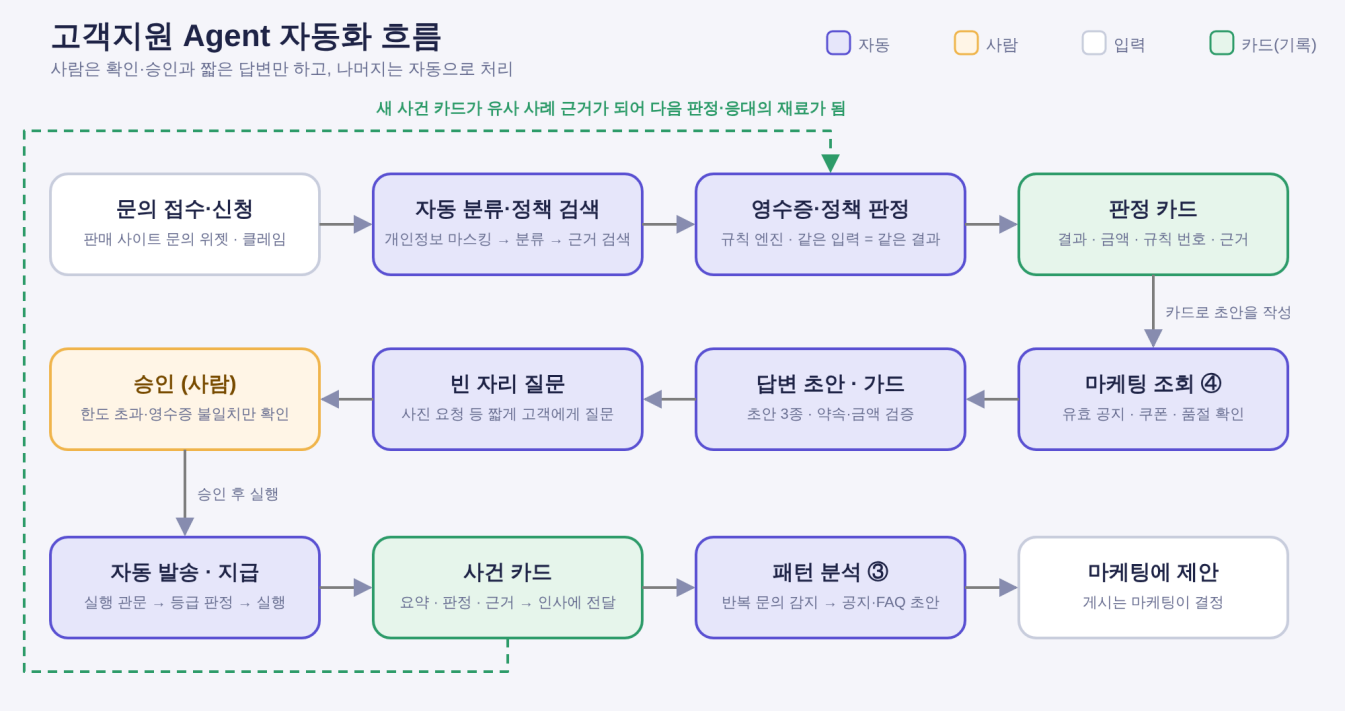
조회	쓰임
유효한 공지·프로모션 목록 (기간·약속 값)	고객이 "공지에서 봤다"고 할 때 근거 확인, 답변 초안에 인용
진행 중 쿠폰·이벤트	**보상 적립금과 이벤트 쿠폰의 중복 지급 방지**, 쿠폰 사용 조건 문의 응대
품절·재입고 상태	재입고 문의 응대 (일정을 약속하지 않고 공지된 사실만 안내)
상품 정보(가격·할인율 변경)	영수증 가격과 현재 가격이 다를 때 설명

- 읽기 전용이므로 등급 대상이 아니다. 근거(조회 시각, 대상 ID)를 결과에 첨부한다 [규약].

- 조회는 **공통 봉투** `{ok, collected_at, data, truncated}` 와 목 모드를 따른다 [규약, common-mcp]. 도구 이름 예: `marketing_get_notices`, `marketing_list_promotions` [제안].
- 보상 지급 직전에 자동 조회하여 중복·충돌이 있으면 `compensation_pay` 를 B 등급 교차검증 단계에서 경고로 올린다.
- 마케팅 DB를 고객지원이 직접 읽지 않는다. 마케팅이 제공하는 API·읽기 전용 뷰로만 읽는다 (규약 "데이터 공유", 모노레포 계약).

---

3. 고객지원 자동화 흐름 (자동 / 사람 / 입력 / 카드)



인사운영 Agent 자동화 흐름과 같은 읽는 법이다. 사람은 **확인·승인과 짧은 답변**만 하고 나머지는 자동이다.

순서	단계	구분	내용	산출물
1	문의 접수·신청	입력	판매 사이트 문의 위젯, 클레임 신청	문의
2	자동 분류·정책 검색	자동	개인정보 마스킹, 분류, 근거 검색 (**[현재 코드]**)	분석 카드
3	영수증·정책 판정	자동	규칙 엔진 (**[현재 코드]**)	**판정 카드**: 결과·금액·규칙 번호·근거
4	마케팅 조회 (④)	자동	유효 공지·쿠폰·품절 확인	공지 근거
5	답변 초안·가드	자동	초안 3종, 약속·금액 검증 (**[현재 코드]**)	초안
6	빈 자리 질문	자동	사진 요청 등 고객에게 짧게 질문 (**[현재 코드]** `needs_info`)	질문

순서	단계	구분	내용	산출물
7	승인	**사람**	한도 초과·영수증 불일치·검수 불일치 등만 확인·승인 (규약상 C 등급 처리 방식은 확인 필요)	승인 기록
8	자동 발송·지급	자동	실행 관문 → 등급 판정 → 발송·지급	실행 결과
9	사건 카드 기록	카드	사건 요약·판정·근거를 카드로 저장, 인사 에이전트에 전달	**사건 카드**
10	패턴 분석 (③)	자동	사건 카드가 쌓여 공지·FAQ 초안의 재료가 됨	제안 초안

사건 카드가 다음 응대의 재료(유사 사례 근거)와 ③의 재료가 되어 고리가 닫힌다.

---

4. 규약 적용

4-1. skill과 등급 (고객지원이 새로 갖는 것)

skill	방향	하는 일	등급	요청 가능 에이전트(ACL, 제안)
`notice_check`	수신 (②)	공지·프로모션 초안 판정	읽기 (등급 없음)	marketing, 통합
`notice_card_register`	수신 (①)	공지 카드 저장	A	marketing
`notice_draft_propose`	발신 (③)	마케팅 제안 큐에 초안 등록 요청	A	— (마케팅의 skill을 호출)
`marketing_get_notices` 등	발신 (④)	유효 공지·프로모션 조회	읽기	— (MCP 또는 마케팅 읽기 skill)

- 마케팅의 `notice_publish` (공지 게시)는 규약 예시대로 **A 등급**이고 마케팅이 신고한다. 쿠폰 지급·대규모 이벤트는 B 이상이 될 수 있으니 재헌이 신고한다.
- 고객지원은 ③에서 제안만 하므로 실행 관문 대상 액션이 없다. ③이 `promises`로 금액을 담는 경우에도 지급이 아니라 제안이다.

4-2. Task 확장 필드 사용

필드	이 루프에서의 쓰임
contextId	공지 하나·패턴 하나가 하나의 사건. ①~④ 모두 같은 contextId
마감 시각	②는 동기 질의이므로 짧게(예: 수 초~수십 초). 초과 시 마케팅은 경고 없이 게시하지 않고 보류 보고
경로	`marketing → support → marketing`처럼 되돌아오는 경로는 루프 거부 대상일 수 있음. ②의 응답은 새 Task가 아니라 같은 Task의 결과로 돌려준다 [제안]
근거	②의 `findings.evidence`, ③의 `evidence`가 이 필드에 들어간다

필드	이 루프에서의 쓰임
untrusted	공지 본문·고객 문의 원문 등 외부 문구

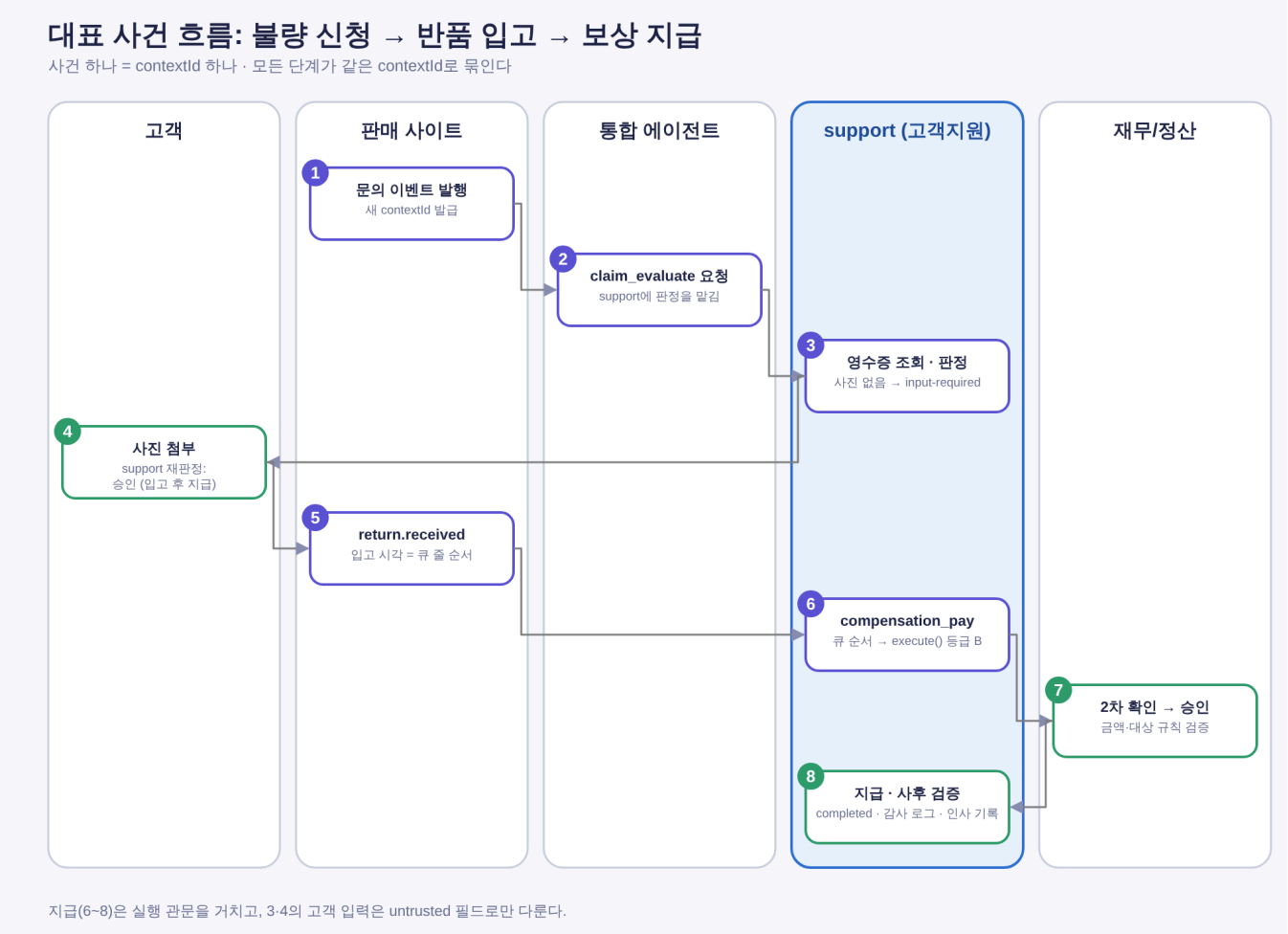
4-3. 해서는 안 되는 것 (README "하지 않는 것"에 추가)

- 고객지원은 공지·프로모션·상품·쿠폰을 직접 바꾸지 않는다.
- 마케팅은 고객지원의 내부 판정 근거(반복 신청 점검 점수 등)를 받지 않는다. ②의 응답에도 포함하지 않는다.
- 공지 문구에서 금액을 정규식으로 추출해 지급 계산에 쓰지 않는다. `promises` 구조화 값만 쓴다.
- 후기 삭제·별점 조건의 보상 공지를 승인하지 않는다 (현재 정책은 후기 삭제 조건만 금지하므로, **별점·내용 조건 보상 금지 조항을 정책 문서에 추가**해야 한다) [제안].

---

5. 대표 시나리오

아래 그림은 구성안의 대표 사건(불량 신청 → 반품 입고 → 보상 지급)이다. 시나리오 A~C는 이 루프(①~④)가 들어가는 경우다.



시나리오 A: 배송 지연 급증 → 공지 → 문의 감소 (③ → ① → ④)

1. 고객지원이 배송 문의가 평소의 6배임을 감지 (코드 규칙).

2. ③ 공지 초안(`promises` 없음)을 마케팅 제안 큐에 등록, 근거로 문의 사례 ID와 집계 쿼리를 첨부.
3. 마케팅이 채택하고, 필요하면 ②로 문구를 점검한 뒤 공지 게시 (A 등급).
4. ① 이벤트·로그가 고객지원에 도착, 공지 카드로 저장.
5. 이후 배송 문의는 ④로 유효 공지를 읽어 답변 초안에 인용한다.
6. 대시보드에서 공지 전후 문의량 변화를 본다.

시나리오 B: 위험한 프로모션 문구를 게시 전에 차단 (② 경고)

1. 마케팅 초안: "지금 사면 무조건 전액 환불".
2. ② `notice_check` → `block`: 단정적 표현, 전자책·주문제작 굿즈 환불 불가 규정과 충돌(`KB-REF-01#3`), 과거 유사 사례 첨부.
3. 마케팅이 경고를 보고 문구를 고쳐 다시 질의 → `ok` → 게시.
4. 만약 무시하고 게시하면 감사 로그와 대시보드에 "block 무시 게시"로 기록된다.

시나리오 C: 쿠폰 이벤트와 보상 중복 방지 (④)

1. 마케팅이 "이번 주 구매자 전원 적립금 1,000원" 이벤트 진행 중.
2. 고객이 배송 지연 보상(적립금 1,000원)을 신청.
3. 보상 지급 전 ④가 진행 중 이벤트를 조회, 이미 같은 목적의 적립금이 지급됐는지 확인.
4. 중복이면 `compensation_pay` 교차검증(B)에서 경고로 올려 재무/정산이 판단.

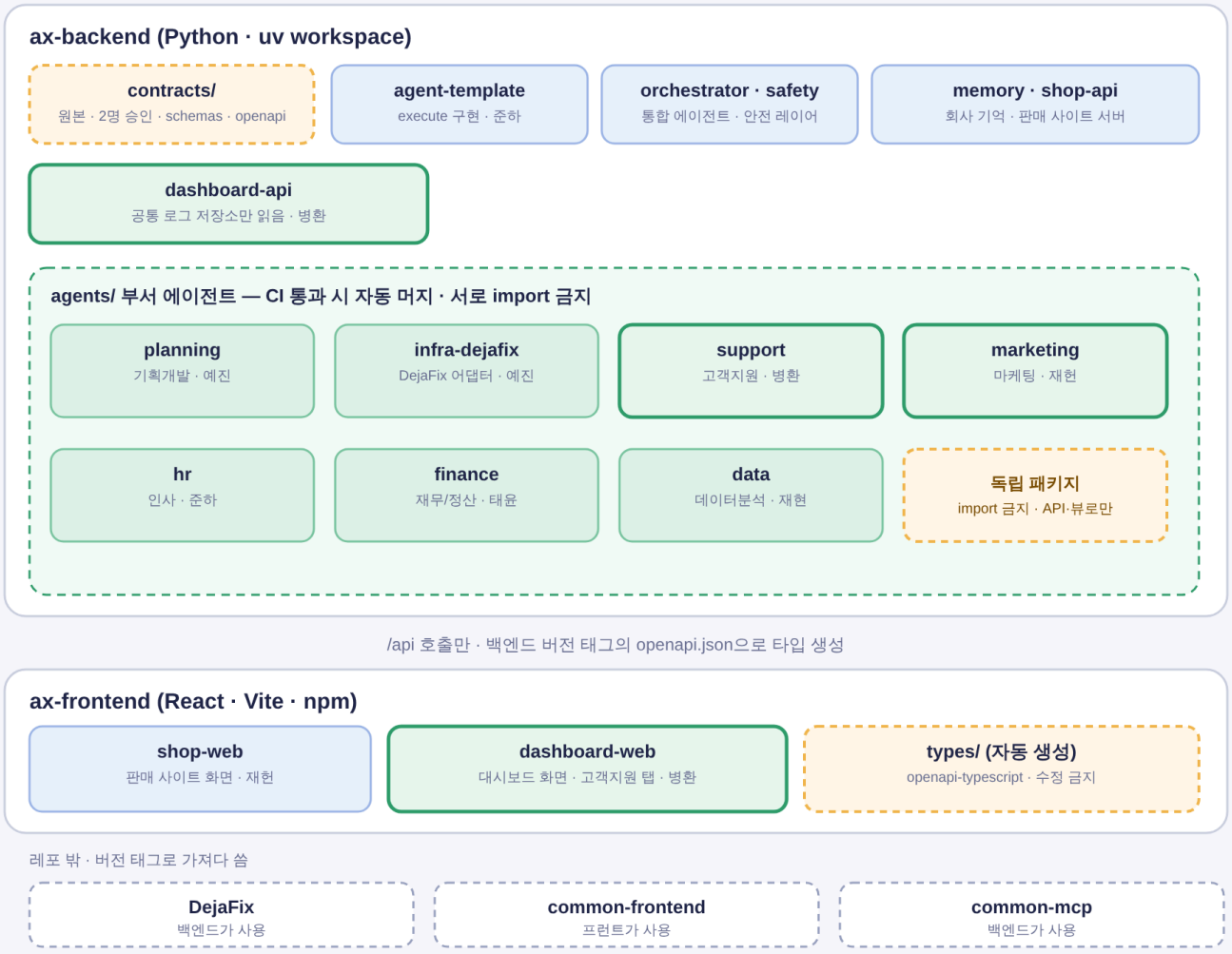
---

6. 저장소와 폴더 배치 (모노레포 + 독립 패키지) [제안]

팀 설계(통합 시연이 핵심이므로 모노레포로 시작하되, 부서 폴더를 독립 패키지처럼 만든다)에 맞춘 고객지원·마케팅 관련 배치다. 이 방식은 아직 결정 기록에 없으므로 합의 후 확정한다.

폴더 배치: 모노레포 + 독립 패키지 (제안)

굵은 초록 = 병환 담당 · 노란 점선 = 합의·승인이 필요한 공유/계약




```
ax-backend/                          Python · uv workspace
├─ contracts/                        원본 · 2명 승인 (schemas · data-access · openapi)
│   └─ schemas/  notice_card.json, notice_check.json, notice_draft.json   ← 이 문서 2장의 스키마
├─ agent-template/                   execute 구현 · 준하
├─ orchestrator/ · safety/ · memory/
├─ shop-api/                         판매 사이트 서버 · 재헌
├─ dashboard-api/                    공통 로그 저장소만 읽음 · 병환
└─ agents/                           CI 통과 시 자동 머지
    ├─ support/                      고객지원 · 병환  ← 독립 패키지
    │   ├─ pyproject.toml · README.md(5항목) · Dockerfile · .env.example
    │   ├─ src/support/  rules/ claims/ queue/ copilot/ kb/ notices/
    │   └─ tests/
    └─ marketing/                    마케팅 · 재헌
ax-frontend/                         React · Vite · npm
├─ dashboard-web/                    대시보드 화면 · 병환 (고객지원 탭, 연동 현황)
└─ types/                            openapi에서 자동 생성 · 수정 금지
```

규칙	고객지원에서 지킬 것
독립 패키지: import 금지, DB는 API·뷰로만	`agents/support`는 `agents/marketing`을 import하지 않는다. 마케팅 데이터는 ④의 API·MCP로만 읽는다
데이터 접근은 계약으로	공지 카드·질의·제안 초안 형식은 `contracts/schemas`에 정의하고 2명 승인을 받는다
대시보드는 공통 로그 저장소만 읽음	`dashboard-api`는 support·marketing의 DB를 직접 보지 않는다. 연동 현황(①~④ 건수, 경고 무시 게시)은 공통 로그로 받는다
CODEOWNERS는 공유 폴더만	`agents/support`는 병환이 소유하고 CI 통과 시 자동 머지. `contracts/`, `agent-template/` 변경은 승인 필요
CI에서 LLM은 mock	현재 프로토타입의 mock 제공자와 테스트가 그대로 맞는다. 실제 호출은 서버 PC 시나리오 테스트에서만
프로젝트 후 `git filter-repo`로 폴더를 이력째 독립 레포로 분리	포트폴리오용으로 `agents/support`를 단독 레포로 뺄 수 있도록 다른 폴더 의존을 만들지 않는다

[현재 코드] 이 저장소의 `backend/app/{rules,claims.py,queue.py,copilot,kb}`가 `agents/support/src/support/`로 이동할 대상이다. `shop.py`, `api_shop.py`, 쇼핑몰 화면은 판매 사이트(재헌)로 가거나 참조 구현으로만 남긴다 (구성안 15장).

---

7. 개발 순서 (이 루프 기준)

단계	할 일	의존
1	공지 카드·질의·초안 스키마를 `contracts/schemas`에 초안으로 작성하고 재헌과 합의	재헌
2	답변 가드를 공지 초안에도 쓰도록 일반화 (`notice_check`의 규칙 부분). 단위 테스트: 단정적 표현, 정책 충돌, 금액 근거	없음 (바로 시작 가능)
3	사례 카드 저장소: 문의·클레임에서 유사 사례를 찾을 색인	없음
4	목 마케팅 클라이언트(④)와 목 이벤트(①)로 단독 테스트	common-mcp 규격
5	패턴 감지 규칙과 ③ 초안 생성 (규칙은 코드, 문장은 LLM)	단계 3
6	마케팅 에이전트와 실제 A2A 연동, 통합 시나리오 A·B·C 리허설	준하(템플릿), 재헌
7	대시보드에 연동 현황 화면 (공지 전후 문의량, 경고 무시 게시, 제안 채택률)	dashboard-api

단계 2·3은 다른 사람의 규격을 기다리지 않고 지금 시작할 수 있다.

---

8. 확인 필요

#	질문	제안
1	② 질의의 `block`을 마케팅이 무시하고 게시할 수 있는가	무시는 허용하되 감사 로그·대시보드에 기록 (판단은 마케팅). 금전 약속이 포함되면 재무/정산 교차검증 필요 여부는 재무/정산과 협의

#	질문	제안
2	②의 응답이 `marketing → support → marketing` 경로를 만들어 규약의 루프 거부에 걸리는가	같은 Task의 결과로 반환 (새 요청 아님). 템플릿(준하) 확인 필요
3	공지 카드의 `promises`를 마케팅이 구조화해서 보내줄 수 있는가	필수 필드로 요청. 없으면 고객지원은 해당 약속을 응대에 쓰지 않음
4	contextId 단위	공지 1개 = 사건 1개. 문의에서 파생된 경우 문의의 contextId를 이어받는지 통합 에이전트와 확인
5	③ 패턴 감지 임계값(N배, N건)	초기값은 고객지원이 제안하고 데이터분석과 조율
6	마케팅·상품MD의 상품 관리 중 어디까지가 ④의 조회 범위인가	가격·할인·품절·재입고 상태까지
7	별점·내용 조건 보상 금지 정책 추가	정책 문서에 조항 추가 (고객지원 소관)
8	모노레포 + 독립 패키지 배치 확정	결정 기록에 한 줄 추가 후 이 문서 6장을 확정으로 변경
9	고객지원 승인 단계(사람)와 C 등급 보류의 관계	구성안 14장 확인 필요 1과 같은 안건